

SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

COURSE NAME: Deep Learning
FACULTY : Dr.Arul Raj A.M

COURSE CODE: MLA0405

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

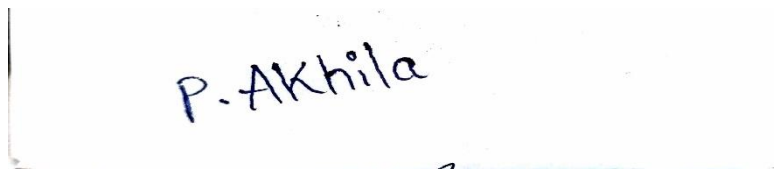
Marks: 25

Code of Conduct:

I **AKHILA.P(192425225)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "P. Akhila".

Name of the student: AKHILA.P

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:

Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer:

Gradient Descent is one of the most widely used optimization algorithms in machine learning. It minimizes the loss function by updating model parameters in the direction of the negative gradient. The **learning rate (η)** determines how large each parameter update is during training.

High Initial Learning Rate

- A high learning rate allows the model to make large updates.
- Training progresses quickly during the initial stages.
- It helps the model escape shallow local regions and move rapidly toward the minimum.
- It reduces the total number of iterations required in the beginning.

However, if the learning rate is extremely high:

- The algorithm may overshoot the minimum.
- Loss values may oscillate.
- Training can become unstable and even diverge.

Learning Rate Decay

Learning rate decay gradually decreases the learning rate as training continues.

Benefits include:

- Smaller and more accurate updates.
- Smooth convergence near the minimum.
- Reduced oscillations.
- Better final accuracy.

When This Strategy Outperforms a Constant Learning Rate

A high initial learning rate followed by decay performs better when:

- Training large datasets.
- Optimizing convex loss functions.
- Training deep neural networks.
- Fast exploration is needed initially and fine-tuning later.

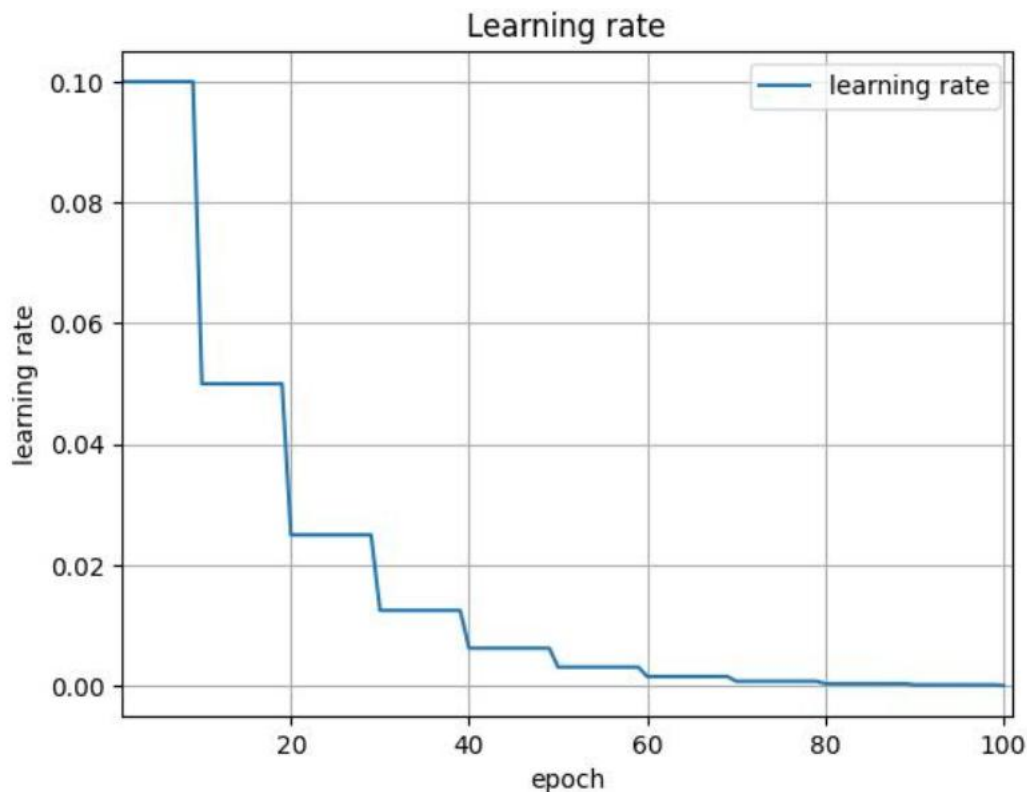
Possible Risks

- Initial learning rate too high $\rightarrow$ divergence.

- Decay too fast → learning stops early.
- Decay too slowly → oscillations continue.

Conclusion

Using a high initial learning rate with gradual decay combines **fast learning** and **stable convergence**, resulting in faster optimization and improved model performance when properly tuned.



2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ , and analytically explain how the estimate changes when:

- The dataset contains outliers
 - The sample size increases
- What does this imply about the robustness of MLE?

Answer

Maximum Likelihood Estimation (MLE) is a statistical method used to estimate unknown parameters by maximizing the likelihood that the observed data were generated from the assumed probability distribution.

For a Gaussian distribution with known variance σ^2 , the likelihood function is:

$$L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}} \quad L(\mu) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i - \mu)^2}{2\sigma^2}}$$

Taking the logarithm and differentiating with respect to μ :

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$$

Thus, the MLE of the mean is simply the **sample mean**.

Effect of Outliers

Outliers are values that differ greatly from the remaining observations.

Their effects include:

- Pulling the estimated mean toward extreme values.
- Increasing estimation error.
- Reducing model accuracy.
- Making the estimate less reliable.

Effect of Increasing Sample Size

As the sample size increases:

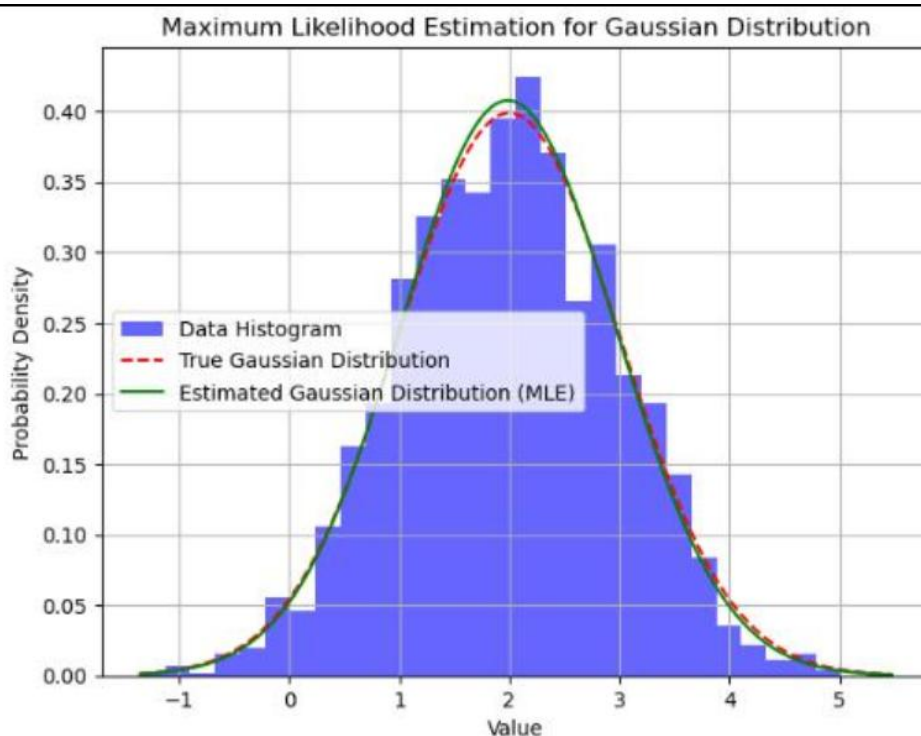
- Estimation variance decreases.
- The estimated mean approaches the true population mean.
- Random fluctuations become smaller.
- The estimate becomes more stable and accurate.

Robustness of MLE

MLE performs well for normally distributed data without extreme values. However, since it depends on the arithmetic mean, it is sensitive to outliers and therefore is **not considered a robust estimator**.

Conclusion

MLE provides efficient and unbiased estimates for clean Gaussian data but loses robustness when datasets contain significant outliers.



3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:

Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer

The dataset contains **10,000 features** but only **500 training samples**, creating a high-dimensional learning problem. This often leads to overfitting because the model has more features than available training examples.

Challenges

- Curse of dimensionality.
- High computational complexity.
- Increased memory requirements.
- Higher risk of overfitting.
- Difficulty identifying useful features.
- Poor prediction on unseen data.

Proposed Algorithm Pipeline

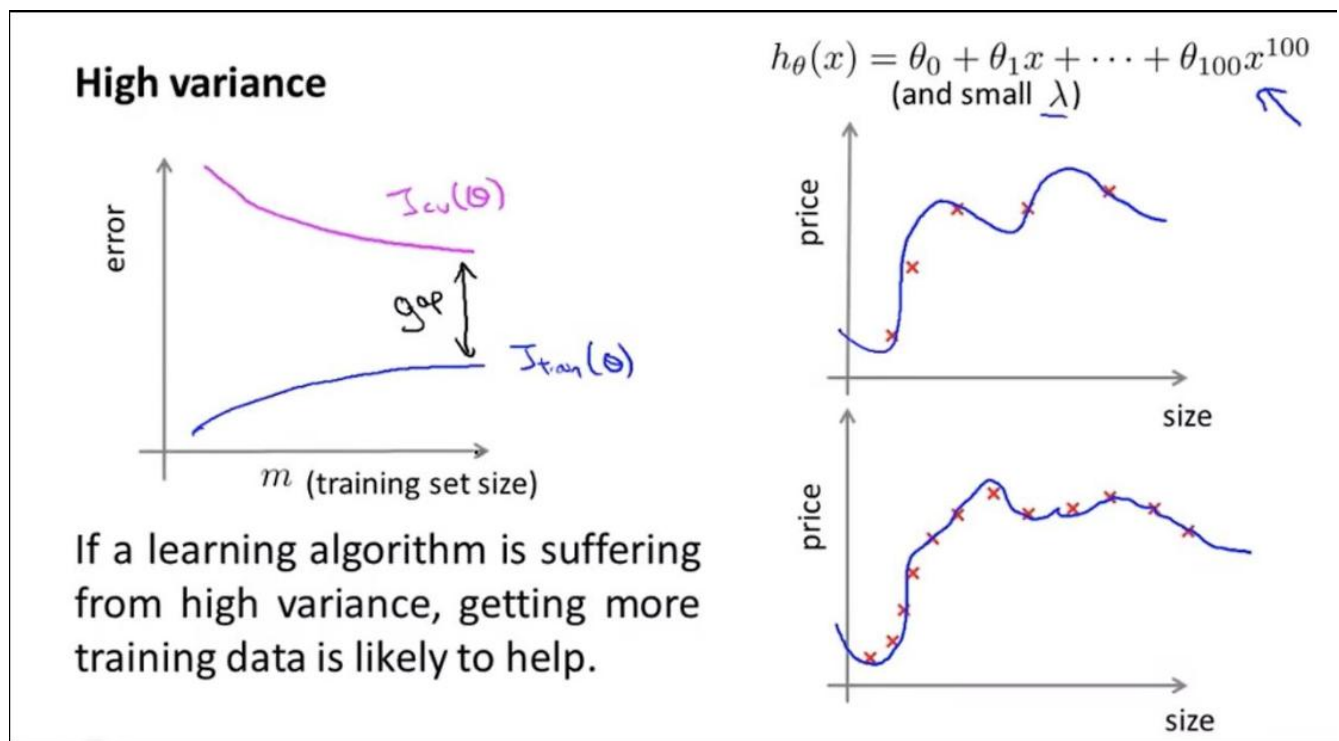
1. **Data Preprocessing**
 - Remove missing values.
 - Normalize features.
 - Eliminate duplicate records.
2. **Feature Selection**
 - Remove irrelevant and redundant features.
 - Keep only informative variables.
 - Reduce model complexity.
3. **Dimensionality Reduction (PCA)**
 - Transform features into principal components.
 - Preserve most important information.
 - Reduce computational cost.
4. **Regularization (L1/L2)**
 - Penalize large model weights.
 - Reduce overfitting.
 - Improve stability.
5. **Model Training**
 - Train an appropriate classifier.
 - Optimize parameters using Gradient Descent.
6. **Cross Validation**
 - Evaluate the model on multiple folds.
 - Improve reliability.
 - Select the best model.

How This Reduces Overfitting

- Fewer features reduce unnecessary complexity.
- Regularization prevents the model from memorizing training data.
- Cross-validation improves generalization.
- PCA removes redundant information.

Conclusion

A combination of preprocessing, feature selection, PCA, regularization, and cross-validation produces a model that is computationally efficient, accurate, and capable of generalizing well to unseen data.



4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:

Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

A Multilayer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers, and an output layer. Each neuron uses an activation function such as the sigmoid function to learn nonlinear relationships.

Why More Hidden Neurons Increase Representational Power

Adding hidden neurons allows the network to:

- Learn more complex mathematical functions.
- Capture nonlinear patterns.
- Represent complicated decision boundaries.
- Improve feature extraction.
- Solve complex classification and regression problems.

According to the Universal Approximation Theorem, an MLP with sufficient hidden neurons can approximate almost any continuous function.

Why More Hidden Neurons Do Not Always Improve Performance

Increasing hidden neurons also introduces several problems:

- Increased overfitting.
- Longer training time.
- Higher computational cost.
- Increased memory usage.
- More difficult optimization.

Overfitting

When the network becomes too large:

- It memorizes the training data.
- Performs very well on training samples.
- Performs poorly on new unseen data.

Generalization

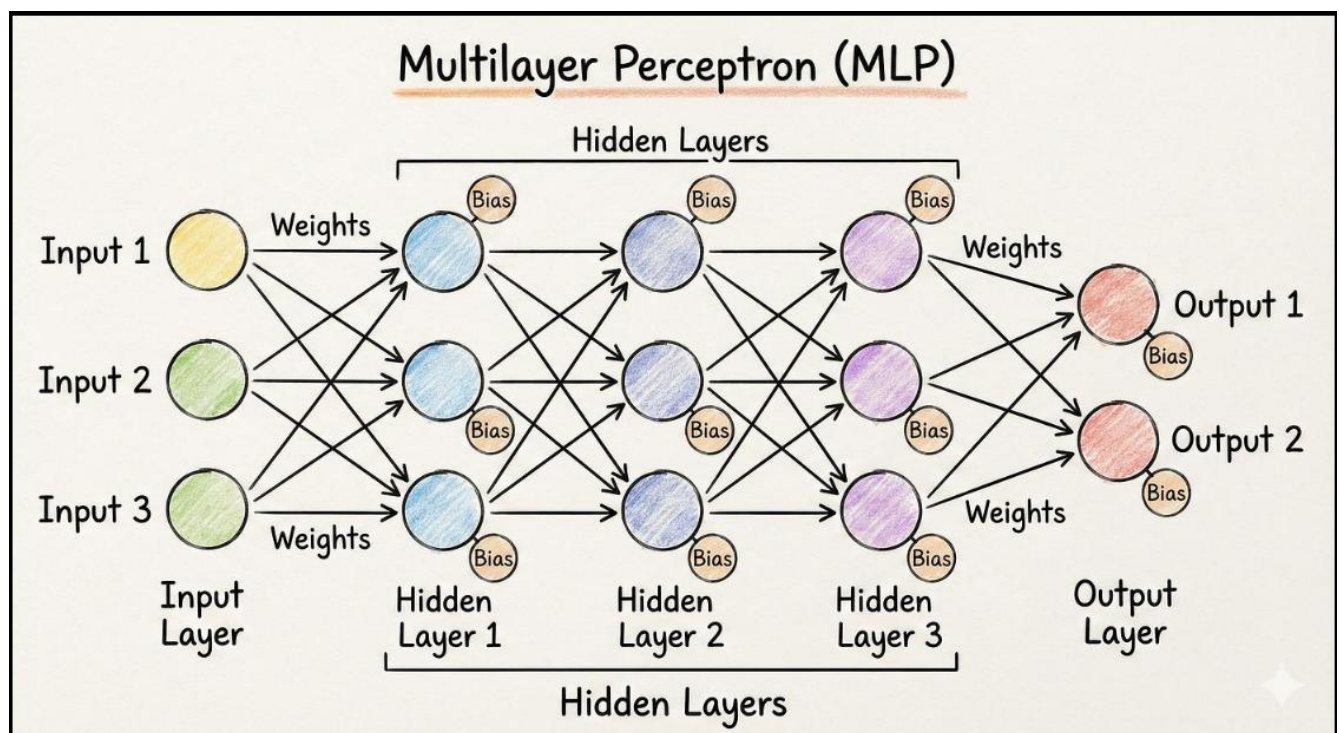
A properly designed MLP should:

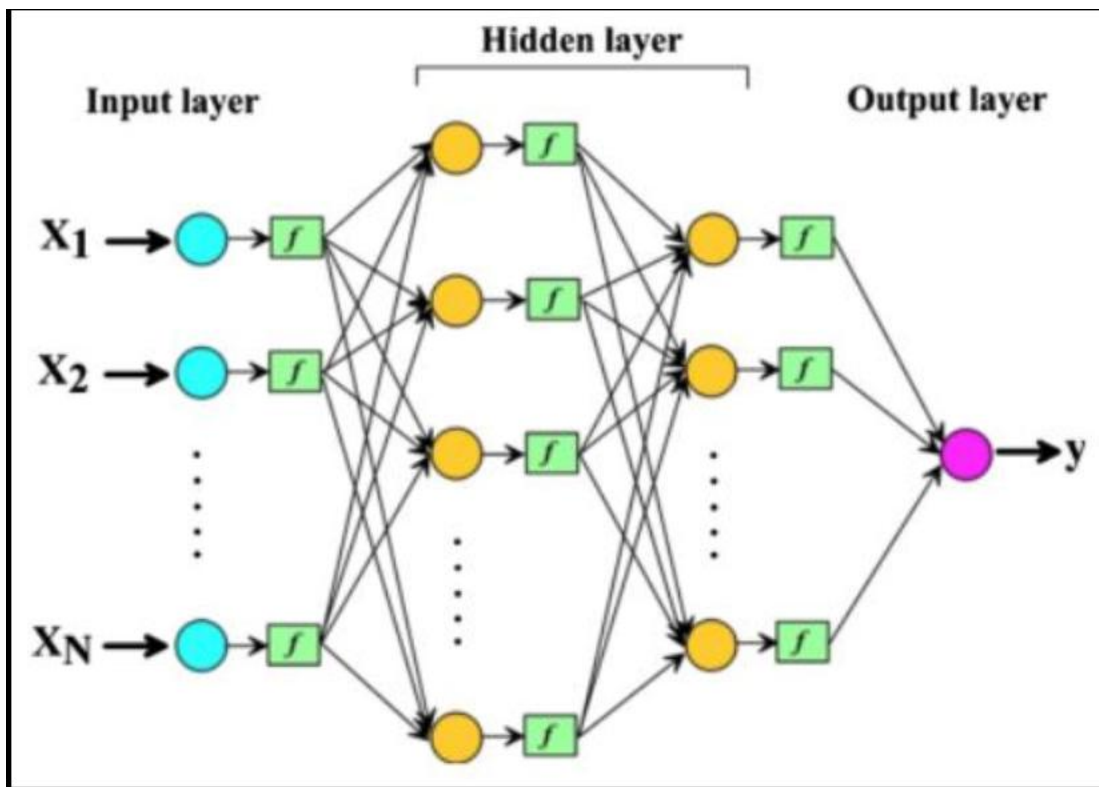
- Learn underlying patterns.
- Ignore random noise.
- Produce accurate predictions on new data.

Regularization, dropout, and early stopping improve generalization.

Conclusion

Adding hidden neurons increases learning capability, but the network size must be carefully selected to balance representational power and generalization.





5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:

Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

The curse of dimensionality refers to problems that arise when the number of features becomes very large. High-dimensional datasets make learning more difficult because data become sparse and distances between samples become less meaningful.

Impact on Gradient Updates in Backpropagation

- More parameters must be updated.
- Gradients become noisy.
- Vanishing or exploding gradients may occur.
- Slower error propagation through the network.
- Increased computational complexity.

Impact on SGD Convergence

- Requires more iterations.
- Training becomes slower.
- Higher variance in gradient estimates.
- Difficult to reach the global optimum.
- Increased training time.

Impact on Model Generalization

- Higher risk of overfitting.
- Poor prediction accuracy on unseen data.
- Reduced reliability.
- Increased model variance.

Techniques to Overcome These Problems

1. Dimensionality Reduction (PCA)

- Removes redundant features.
- Lowers computational cost.
- Improves convergence speed.
- Preserves important information.

2. Regularization (L1/L2 or Dropout)

- Controls model complexity.
- Prevents overfitting.
- Improves generalization.

Additional Techniques

- Feature Selection.
- Batch Normalization.
- Early Stopping.
- Data Augmentation.
- Increasing training data.

Conclusion

The curse of dimensionality negatively affects backpropagation, SGD convergence, and model generalization. Techniques such as dimensionality reduction, feature selection, and regularization significantly improve learning efficiency and prediction accuracy.

